LOW-LEVEL DESIGN (LLD)

RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop (HITL)

Prepared by: Kirthika R S

Internship Project — 2025

1. Module-Level Design

The system is divided into the following modules, each responsible for a specific function in the pipeline:

1.1 Document Processing Module

File: ingest.py

- Loads the PDF knowledge base using PyPDFLoader from LangChain Community
- Extracts raw text from every page of the document
- Passes the extracted text to the Chunking Module for further processing
- Entry point for the entire data ingestion pipeline

1.2 Chunking Module

Tool: RecursiveCharacterTextSplitter

- Splits extracted text into smaller overlapping chunks
- Chunk size is set to 500 characters for balanced context
- Chunk overlap is set to 50 characters to avoid losing context at boundaries
- Recursive splitting respects sentence and paragraph boundaries

1.3 Embedding Module

Model: all-MiniLM-L6-v2 (HuggingFace Sentence Transformers)

- Converts each text chunk into a 384-dimensional numerical vector
- Embeddings capture semantic meaning of the text
- Runs entirely locally — no API key or internet required
- Same model is used at both ingestion time and query time

1.4 Vector Storage Module

Tool: FAISS (Facebook AI Similarity Search)

- Stores all chunk embeddings in a local FAISS index
- Index is saved to disk in the vectorstore/ folder
- Supports fast approximate nearest neighbor search
- Loaded from disk at query time for retrieval

1.5 Retrieval Module

File: retriever.py

- Loads the FAISS index from disk
- Converts the user query into an embedding using the same model
- Searches the FAISS index for top-3 most similar chunks
- Returns the matching chunks as a combined context string

1.6 Query Processing Module

File: graph.py — process_node function

- Receives the user query as input
- Calls the Retrieval Module to fetch relevant context
- Evaluates the retrieved context length to decide routing
- Sets the escalate flag in the state if context is insufficient

1.7 Graph Execution Module

File: graph.py — build_graph function

- Builds a LangGraph StateGraph with three nodes: process, answer, escalate
- Defines conditional edges based on the escalate flag in the state
- Compiles and returns the executable graph
- main.py calls graph.invoke() to run the full workflow

1.8 HITL Module

File: hitl.py

- Triggered when the escalate flag is True
- Prints escalation message and the original customer query
- Accepts a manual response typed by a human agent
- Returns the human response as the final answer

2. Data Structures

2.1 Document Representation

LangChain Document Object:

```
Document(page_content='raw text from PDF page', metadata={'source':
'data/knowledge_base.pdf', 'page': 0})
```

2.2 Chunk Format

Each chunk is also a LangChain Document object:

```
Document(page_content='Q: How do I reset my password? A: Go to
Settings...', metadata={'source': 'data/knowledge_base.pdf', 'page':
0})
```

2.3 Embedding Structure

Each chunk is converted to a float32 numpy array of shape (384,):

```
embedding = model.encode('chunk text')  # shape: (384,) float32 array
```

2.4 Query-Response Schema

Field	Type	Description
query	str	Raw user question from CLI input
context	str	Top-3 retrieved chunks joined by newline
answer	str	Final answer from LLM or human agent
escalate	bool	True if context is insufficient for bot to answer

2.5 State Object for LangGraph

The state is a TypedDict passed between all nodes in the graph:

```
class State(TypedDict):
    query: str        # User input question
    context: str      # Retrieved document chunks
    answer: str       # Final generated answer
    escalate: bool    # Routing flag for HITL
```

3. Workflow Design (LangGraph)

3.1 Nodes

Node Name	Function	Responsibility
process	process_node()	Retrieves context from FAISS, sets escalate flag
answer	answer_node()	Sends context + query to Groq LLM, gets answer
escalate	escalate_node()	Prompts human agent to provide manual answer

3.2 Edges

- Entry point: process node
- process → answer (when escalate is False)
- process → escalate (when escalate is True)
- answer → END
- escalate → END

3.3 State Transitions

Initial State: { query: user_input, context: '', answer: '', escalate: False }

After process node: context is filled, escalate is set True or False

After answer node: answer is filled with LLM response

After escalate node: answer is filled with human agent response

4. Conditional Routing Logic

4.1 Answer Generation Criteria

- Retrieved context must be at least 20 characters long
- If context length >= 20 characters → route to answer node
- LLM is given the context and query and asked to generate a helpful response
- If LLM is unsure, it is instructed to say it does not know

4.2 Escalation Criteria

- Low confidence: Retrieved context is empty or shorter than 20 characters
- Missing context: No matching chunks found in FAISS for the query
- Complex query: Query does not match any FAQ in the knowledge base

Routing function code logic:

```
def route(state):
    if state['escalate']:
        return 'escalate'
    return 'answer'
```

5. HITL Design

5.1 When Escalation is Triggered

- The process_node checks the length of the retrieved context string
- If context is empty or less than 20 characters, escalate is set to True
- The route() function reads the escalate flag and directs flow to escalate node

5.2 What Happens After Escalation

- The system prints a clear escalation message to the terminal
- The original customer query is displayed to the human agent
- The human agent types a manual response into the terminal
- The typed response is captured using Python input() function

5.3 How Human Response is Integrated

- The human response string is stored in state['answer']
- This answer is returned to main.py just like a bot-generated answer
- The user sees the human response as the final bot reply
- No distinction is shown to the user between bot and human responses

6. API / Interface Design
7.

6.1 Input Format

The system accepts plain text input from the user via the terminal (CLI):

```
You: How do I reset my password?
```

6.2 Output Format

The system outputs a plain text answer to the terminal:

```
Bot: To reset your password, go to Settings > Account > Reset Password.
     Enter your registered email and click Send OTP.
```

6.3 Interaction Flow

- Step 1: User runs python main.py
- Step 2: Welcome message is displayed
- Step 3: User types a query and presses Enter
- Step 4: System searches knowledge base and generates answer
- Step 5: Answer is displayed as Bot response
- Step 6: User can type another query or type quit to exit

7. Error Handling

7.1 Missing Data

- If the PDF file is missing, PyPDFLoader raises a FileNotFoundError
- The system should be run only after placing the PDF in the data/ folder
- Running create_pdf.py ensures the knowledge base PDF exists

7.2 No Relevant Chunks Found

- If FAISS returns no relevant chunks, context will be an empty string
- The process_node detects this and sets escalate = True
- The query is routed to the HITL module for human handling
- This prevents the LLM from hallucinating answers without context

7.3 LLM Failure

- If Groq API is unavailable, an exception is raised in answer_node
- The system should wrap LLM calls in try-except blocks for production use
- A fallback message like 'Service temporarily unavailable' can be returned
- Rate limit errors from Groq free tier should be handled with retry logic

--- End of LLD Document ---